

```
$ npx skills add addyosmani/agent-skills
```

Agent Skills

Teaching AI coding agents to work like a senior engineer.

Tejas — Senior Java Architect

[EVENT / DATE]

github.com/addyosmani/agent-skills · MIT

AI agents write code fast. They also skip the boring, important parts.

SKIPPED BY DEFAULT

Specs

Tests

Security reviews

Everything that makes software reliable

WHAT A SKILL ADDS

A step-by-step recipe the agent must follow, with a clear "done" check at the end. Like a Definition of Done, but for your AI.

25

skills

9

slash commands

4

reviewer personas

7

reference checklists

Same steps you already follow — now the agent follows them too



You don't always type a command. Skills switch on by themselves: designing an API pulls in `api-and-interface-design`, building UI pulls in `frontend-ui-engineering`.

A skill is a recipe file the agent reads and follows

```
skills/test-driven-development/SKILL.md
```

name + description	Tells the agent when to use it
-----------------------	-----------------------------------

Overview	What this skill does
----------	----------------------

When to Use	The trigger conditions
-------------	------------------------

Process	The steps, in order
---------	---------------------

Rationalizations	Common excuses + why they're wrong
------------------	---------------------------------------

Red Flags	Signs something's going wrong
-----------	-------------------------------

Verification	Proof required before calling it done
--------------	---------------------------------------

It's a plain Markdown file. No SDK, no framework.

Think of it as a **runbook** written for the agent instead of for a person.

Every skill has the same shape, so the agent always knows where to look.

Nine commands. That's the whole interface.

TYPE THIS	WHEN	THE RULE BEHIND IT
<code>/spec</code>	Decide what to build	Spec before code
<code>/plan</code>	Plan how to build it	Small, atomic tasks
<code>/build</code>	Build step by step	One slice at a time
<code>/test</code>	Prove it works	Tests are proof
<code>/constraints</code>	Set the quality bar	Decide once, enforce everywhere
<code>/review</code>	Review before merge	Improve code health
<code>/webperf</code>	Audit web performance	Measure before you optimize
<code>/code-simplify</code>	Make code simpler	Clarity over cleverness
<code>/ship</code>	Release to production	Faster is safer

25 skills, grouped by when you need them

DEFINE · 4

interview-me
idea-refine
spec-driven-development
constraint-driven-development

PLAN · 1

planning-and-task-breakdown

BUILD · 7

incremental-implementation
test-driven-development
context-engineering
source-driven-development
doubt-driven-development
frontend-ui-engineering
api-and-interface-design

VERIFY · 2

browser-testing-with-devtools
debugging-and-error-recovery

REVIEW · 4

code-review-and-quality
code-simplification
security-and-hardening
performance-optimization

SHIP · 6

git-workflow-and-versioning
ci-cd-and-automation
deprecation-and-migration
documentation-and-adrs
observability-and-instrumentation
shipping-and-launch

Plus one meta-skill, `using-agent-skills`, that picks the right skill for the task.

Agents make excuses. Skills answer them in advance.

THE AGENT THINKS

"I'll add tests later."

"It seems right."

"It's a small change, skip the spec."

THE SKILL ANSWERS

Tests are the proof. No test, not done.

Not enough. Show passing tests, build output or runtime data.

Spec before code: a short spec is still a spec.

Every skill ships a table of these excuses with a counter-argument, and ends with the evidence it needs before it may say "done".

Battle-tested ideas, in plain words

Hyrum's Law

If users can see a behaviour, someone depends on it.

→ API design

Beyonce Rule

If it matters, put a test on it.

→ Testing

Test pyramid 80/15/5

Mostly unit tests, some integration, few end-to-end.

→ Testing

~100-line changes

Small changes get real reviews.

→ Code review

Chesterton's Fence

Know why code exists before you delete it.

→ Simplification

Trunk-based development

Short branches, merge often.

→ Git workflow

Shift Left

Catch problems as early as possible.

→ CI/CD

Feature flags

Ship code switched off, turn it on safely.

→ CI/CD · Launch

Code is a liability

Less code to keep alive is better.

→ Deprecation

Built-in reviewers and checklists

4 REVIEWER PERSONAS

`code-reviewer`

Senior Staff Engineer — "Would a staff engineer approve this?"

`test-engineer`

QA Specialist — Test strategy and coverage

`security-auditor`

Security Engineer — Threat modelling, OWASP

`web-performance-auditor`

Web Perf Engineer — Core Web Vitals, via /webperf

7 CHECKLISTS, LOADED ONLY WHEN NEEDED

`definition-of-done.md`

`testing-patterns.md`

`security-checklist.md`

`performance-checklist.md`

`accessibility-checklist.md`

`observability-checklist.md`

`orchestration-patterns.md`

09 · Try it

Install in one command

ANY AGENT (70+ SUPPORTED)

```
$ npx skills add addyosmani/agent-skills
```

CLAUDE CODE

```
/plugin marketplace add addyosmani/agent-skills  
/plugin install agent-skills@addy-agent-skills
```

Also native for Codex, Cursor, Gemini CLI, Copilot, OpenCode and more.

/build auto

You approve the plan once. The agent then builds every task by itself.

Still test-first. Still one commit per task. It stops when something fails or looks risky.

Two ways to start

NEW PROJECT

Use the full lifecycle from day one.

EXISTING CODEBASE

Roll out step by step, verification first.

Java teams: skills are language-neutral Markdown, but the testing-patterns examples are JavaScript/TypeScript. Expect to adapt those to JUnit.

Questions?